SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I **MATHIMZOZHI – 192321121 (**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Answer:

1. Formulation as a Constraint Satisfaction Problem (CSP)

- **Variables:** D1, D2, D3 (the shift of each doctor).
- **Domain of each variable:** {Morning (M), Afternoon (A), Night (N)}, ordered M < A < N.
- **Constraints:** (i) D1 ≠ N; (ii) shift(D2) < shift(D3); (iii) all three doctors have different shifts (one doctor per shift); (iv) D3 ≠ M; (v) every doctor is assigned exactly one shift (a complete assignment).

2. Backtracking Search

Backtracking is Depth-First Search that assigns one variable at a time and checks the constraints after every assignment. If a constraint is broken, it **backtracks** (undoes the last assignment and tries the next value). Variable order: D1, D2, D3. Value order: Morning, Afternoon, Night.

```
function BACKTRACK(assignment)
    if assignment is complete then return assignment
    var <- next unassigned variable (D1, D2, D3)
    for each value in (Morning, Afternoon, Night) do
        if value is consistent with assignment (all constraints) then
            add (var = value) to assignment
            result <- BACKTRACK(assignment)
            if result is not failure then return result
            remove (var = value) from assignment      // backtrack
    return failure
```

3. Intermediate steps and constraint checks

Step	Assignment tried	Constraint check	Result
1	D1 = Morning	(i) D1 ≠ Night: satisfied	Consistent, go deeper
2	D2 = Morning	(iii) violated: Morning already taken by D1	**Backtrack**, try next value
3	D2 = Afternoon	(iii) satisfied (M, A are different)	Consistent, go deeper
4	D3 = Morning	(iv) violated (D3 ≠ Morning) and (iii) violated	**Backtrack**
5	D3 = Afternoon	(iii) violated: Afternoon already taken by D2	**Backtrack**

Step	Assignment tried	Constraint check	Result
6	D3 = Night	(iii) satisfied; (iv) D3 ≠ Morning satisfied; (ii) Afternoon < Night satisfied; (i) satisfied; (v) all 3 doctors assigned	**All constraints satisfied: SOLUTION**

4. Final valid schedule

Shift	Doctor
Morning	**D1**
Afternoon	**D2**
Night	**D3**

5. Example check of the final answer

Constraint	Check with D1 = M, D2 = A, D3 = N	Satisfied?
(i) D1 cannot work Night	D1 works Morning	Yes
(ii) D2 works before D3	Afternoon < Night	Yes
(iii) One doctor per shift	M, A, N each used once	Yes
(iv) D3 cannot work Morning	D3 works Night	Yes
(v) Everyone gets exactly one shift	D1, D2, D3 each have one shift	Yes

Note: the search stops at the first solution. If it continued, it would also find D1 = Afternoon, D2 = Morning, D3 = Night, which satisfies the same constraints. (D1 = Night is rejected at once by rule (i), and D3 = Afternoon always leads to D2 = Morning and D1 = Night, which fails.) So this problem has exactly **2 valid schedules**, and the first one found is shown above.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

i. Movement allowed: Up, Down, Left, Right
ii. Cost of each move = 1
iii. Cannot pass through obstacles
iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

Answer:

Assumption: the question does not include the grid picture, so a sample 5×5 grid is used. Cells are written as (row, column), counted from 0 at the top-left. **S = (0,0)**, **G = (4,4)**. The dark cells in Fig. 2 are the obstacles: (0,1), (1,3), (2,0), (2,1), (2,3), (4,2). If your paper has a different grid, only the numbers change; the steps are the same.

1. Problem formulation

- **State:** the robot's cell (row, column). **Initial state:** S. **Goal test:** cell = G.
- **Actions:** Up, Down, Left, Right (not into an obstacle or outside the 5×5 grid). **Cost:** 1 per move.
- **Manhattan distance heuristic:** h(n) = |row − 4| + |col − 4|. For example h(S) = 4 + 4 = 8, h(4,3) = 1, h(G) = 0.

2. BFS: step-by-step node expansion (FIFO queue)

BFS removes the **oldest** node from the queue, adds its unvisited neighbours at the back (order: Up, Down, Left, Right) and stops when G is removed. Because every move costs 1, the first time G is reached the path is optimal.

Step	Node expanded	Children added to the queue	Queue after the step
1	(0,0)	(1,0)	(1,0)
2	(1,0)	(1,1)	(1,1)
3	(1,1)	(1,2)	(1,2)
4	(1,2)	(0,2), (2,2)	(0,2), (2,2)
5	(0,2)	(0,3)	(2,2), (0,3)
6	(2,2)	(3,2)	(0,3), (3,2)
7	(0,3)	(0,4)	(3,2), (0,4)
8	(3,2)	(3,1), (3,3)	(0,4), (3,1), (3,3)
9	(0,4)	(1,4)	(3,1), (3,3), (1,4)
10	(3,1)	(4,1), (3,0)	(3,3), (1,4), (4,1), (3,0)
11	(3,3)	(4,3), (3,4)	(1,4), (4,1), (3,0), (4,3), (3,4)
12	(1,4)	(2,4)	(4,1), (3,0), (4,3), (3,4), (2,4)
13	(4,1)	(4,0)	(3,0), (4,3), (3,4), (2,4), (4,0)
14	(3,0)	–	(4,3), (3,4), (2,4), (4,0)
15	(4,3)	(4,4)	(3,4), (2,4), (4,0), (4,4)
16	(3,4)	–	(2,4), (4,0), (4,4)
17	(2,4)	–	(4,0), (4,4)
18	(4,0)	–	(4,4)
19	(4,4)	–	–

3. With the Manhattan heuristic: priority queue updates

The question also mentions a priority queue and the Manhattan heuristic. BFS itself does not use a heuristic, so for completeness here is the same grid solved with a **priority queue ordered by f(n) = g(n) + h(n)** (the A* method). g = cost so far, h = Manhattan distance, f = g + h. Only 10 nodes are expanded instead of 19.

Step	Node expanded	g / h / f	Children pushed	Priority queue after step (cell : f)
1	(0,0)	0 / 8 / 8	(1,0)	(1,0):8
2	(1,0)	1 / 7 / 8	(1,1)	(1,1):8
3	(1,1)	2 / 6 / 8	(1,2)	(1,2):8
4	(1,2)	3 / 5 / 8	(0,2), (2,2)	(2,2):8, (0,2):10
5	(2,2)	4 / 4 / 8	(3,2)	(3,2):8, (0,2):10
6	(3,2)	5 / 3 / 8	(3,1), (3,3)	(3,3):8, (0,2):10, (3,1):10
7	(3,3)	6 / 2 / 8	(4,3), (3,4)	(3,4):8, (4,3):8, (0,2):10, (3,1):10
8	(3,4)	7 / 1 / 8	(2,4), (4,4)	(4,3):8, (4,4):8, (0,2):10, (3,1):10, (2,4):10
9	(4,3)	7 / 1 / 8	–	(4,4):8, (0,2):10, (3,1):10, (2,4):10
10	(4,4)	8 / 0 / 8	–	(0,2):10, (3,1):10, (2,4):10

4. Optimal path and cost

Optimal path (BFS): (0,0) → (1,0) → (1,1) → (1,2) → (2,2) → (3,2) → (3,3) → (4,3) → (4,4).

Number of moves = 8, so the total cost = 8 × 1 = 8. The heuristic search finds another path of the same cost, (0,0) → (1,0) → (1,1) → (1,2) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4), so both are optimal. The cost 8 also equals h(S) = 8, so no path can be shorter.

Interpretation: BFS is guaranteed optimal here because all costs are equal, but it expanded 19 of the 19 reachable cells, nearly the whole grid. The Manhattan heuristic guides the search toward G and expanded only 10 cells.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

Answer:

Each constraint limits what the robot can do or know, and together they decide how it behaves:

Constraint	How it affects decision-making	Priority
Blocked cells (obstacles)	Hard constraint: the robot may never enter them, so they cut the action set and can make a planned route impossible. Safety comes first.	1 (highest)
Limited sensing (adjacent cells only)	The robot never has the full map (partial observability). It must plan with incomplete knowledge, treat unseen cells as unknown and keep sensing as it moves.	2
Dynamic knowledge update (online search)	Plans become outdated when new cells are discovered, so the robot must repeat sense, update, replan after every move instead of planning once.	3
Move cost 1, risky zone +2	Entering a risky cell costs 1 + 2 = 3. It is allowed but expensive, so a risky cell is worth entering only if the safe detour is more than 2 steps longer.	4
Minimise total cost with safe navigation	The objective: the cheapest safe route, not the shortest in steps. This is why a cost-sensitive search (Uniform Cost Search) is needed instead of BFS.	5 (goal)

Example: the robot stands at (2,0) and finds (3,0) blocked and (2,1) risky. Going through the risky cell costs 3 + 1 + 1 + 1 + 1 + 1 = 8 to reach G, while going back and around costs 10. Because the risk penalty is only +2, the robot rationally accepts the risky cell.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

Answer:

Chosen strategy: Online Uniform Cost Search with replanning

The robot uses **Uniform Cost Search (UCS)** on the part of the map it knows, and repeats it after every move, which makes it an **online search agent**. UCS always expands the cheapest path first, so it handles the unequal costs (1 or 3) correctly.

Steps

- **Step 1: Sense.** Observe the four adjacent cells and classify each as free, blocked or risky.

- **Step 2: Update the knowledge map.** Remember every cell seen. Cells never seen are optimistically assumed free (cost 1).
- **Step 3: Plan.** Run UCS from the current cell to G on the known map. Blocked cells are removed, a risky cell costs 3.
- **Step 4: Act.** Execute only the first move of the plan.
- **Step 5: Repeat** from the new cell. Stop when G is reached and rescue the survivor.

Fig. 3: Sense, update, plan and act loop of the online agent

```
function ONLINE-UCS-AGENT(start, goal)
    known <- empty map;  pos <- start
    while pos != goal do
        SENSE the adjacent cells and add them to known
        plan <- UCS(known, pos, goal)   // blocked = removed, risky = cost 3,
                                        // unknown cells = assumed free (cost 1)
        if plan = failure then return "no safe path known"
        MOVE one step along plan;  pos <- new cell
    return "survivor reached"
```

Worked example

Assumed 5×5 building (not given in the question): S = (0,0), G = (4,4), blocked cells = (1,1), (1,4), (2,3), (3,0), (3,2); risky cells = (1,3), (2,1), (3,4). The robot does not know this layout in advance; it learns it only by sensing adjacent cells.

Robot at	Newly sensed (adjacent cells)	Decision for the next move (estimated cost to G)	Cost so far
S (0,0)	(1,0) free, (0,1) free	Plan with unknown cells free: go Down (cost-to-go 8)	0
(1,0)	(1,1) blocked, (2,0) free	Continue Down (cost-to-go 7)	1
(2,0)	(3,0) blocked, (2,1) **risky**	Replan: through the risky cell = 8, back-and-around = 10, so go Right through (2,1)	2
(2,1)	(3,1) free, (2,2) free	Move cost 3 (1 + 2 risk). Go Down (cost-to-go 5)	5
(3,1)	(4,1) free, (3,2) blocked	Continue Down (cost-to-go 4)	6
(4,1)	(4,0) free, (4,2) free	Go Right (cost-to-go 3)	7
(4,2)	(4,3) free	Go Right (cost-to-go 2)	8
(4,3)	G seen, (3,3) free	Go Right (cost-to-go 1)	9
G (4,4)	–	**Survivor reached**	**10**

Fig. 4: The assumed building and the path actually travelled by the robot

Result: path S → (1,0) → (2,0) → (2,1) → (3,1) → (4,1) → (4,2) → (4,3) → G with **total cost 10** (8 moves; one of them enters a risky cell, 8 + 2 = 10). It never entered a blocked cell. With a full map in advance, offline UCS also gives cost 10, so here the online agent did as well as an agent with complete knowledge.

c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

Answer:

1. Intelligent agent

The robot is an **intelligent agent**: it perceives its environment through sensors and acts on it through actuators to achieve a goal. It is a **model-based, utility-based agent**: it keeps an internal map (model), has a goal (reach G) and prefers actions of lower cost.

PEAS	Rescue robot
Performance	Reach the survivor, minimum total cost (1 per move + 2 per risky entry), no entry into blocked cells, safety
Environment	Grid of rooms with blocked and risky cells, partially known
Actuators	Wheels or tracks (Up, Down, Left, Right moves)
Sensors	Proximity or camera sensors for the 4 adjacent cells (free, blocked, risky)

2. Rationality

A rational agent selects the action that is expected to maximise its performance measure, based on its percept sequence and built-in knowledge. Rational does not mean all-knowing: the robot cannot know unseen cells, but it does the best possible with what it has sensed.

- It never enters blocked cells (safety first).
- It compares costs before choosing: in the example it paid the +2 risk penalty at (2,1) because the safe detour cost 10 against 8.
- It replans whenever new percepts change the best plan, which is how it improves its decisions as knowledge grows.

3. Environment type

Property	Rescue environment	Reason
Observable	Partially observable	Sees only adjacent cells
Agents	Single agent	Only the robot acts
Deterministic?	Deterministic	A move succeeds and leads to the intended cell
Episodic?	Sequential	Each move affects later cost and position
Static?	Static (assumed)	The layout does not change during the search; if rubble could shift, it would be dynamic
Discrete?	Discrete	Finite grid cells and four actions
Known?	Unknown (initially)	The map is learned during the mission, hence online search

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Answer:

Online UCS with replanning is the most suitable approach for these constraints:

Strategy	Why it is not enough
BFS	Counts steps only and ignores the extra risky-zone cost, so it may choose an expensive risky route
DFS / greedy search	Not optimal and may wander into risky zones or long detours
Offline UCS / A*	Needs the complete map before moving, which is impossible with sensing limited to adjacent cells
Online UCS with replanning	Handles unequal costs, uses only what is sensed, and updates the plan after every move

- **Cost-sensitive and optimal on known data:** UCS always finds the cheapest path in the known map, so the +2 risk penalty is balanced against detour length.
- **Safe:** blocked cells are removed from the search space, so the robot never plans through them.
- **Handles partial observability:** the knowledge map is updated after each move and a new plan is made immediately.
- **Complete:** on a finite grid it always finds a route if one exists, because it keeps expanding until the goal is reached.

- **Simple and practical:** no prediction of hidden cells is needed, so it can run on a small robot.

Limitation and improvement: in an unknown world no online method can guarantee the globally cheapest route, because a plan may need correction after new cells are seen. Adding the Manhattan distance as a heuristic (A* style replanning, as in D* Lite) keeps the same safety and makes the search faster.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided